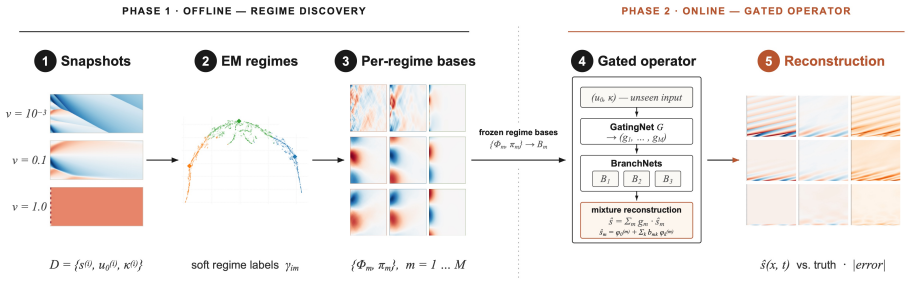
# Graphical Abstract

## TEMPO: Regime-Aware Operator Learning with Locally Adapted POD Bases

Anonymous Authors



# Highlights

**TEMPO: Regime-Aware Operator Learning with Locally Adapted POD Bases**

Anonymous Authors

- TEMPO discovers solution regimes from data and fits a dedicated basis to each.
- Regime-local bases match or outperform a single global basis of the same rank.
- TEMPO(POD) reduces error by 13–50% on Burgers' and up to 5× on Darcy flow.
- TEMPO trains across all parameter values and handles multiple solution regimes.
- Regime structure reflects initial conditions and physical parameters without labels.

# TEMPO: Regime-Aware Operator Learning with Locally Adapted POD Bases

Anonymous Authors

*Anonymous Institution,*

**Abstract**

Operator learning approximates the solution map of a parametric PDE from data, enabling fast prediction at new parameter values without repeated solves. The dominant approach fits a single global low-rank basis to all solution snapshots, an assumption that fails when the governing parameter drives qualitatively distinct solution regimes: smooth diffusive profiles versus near-discontinuous shocks, or solutions differing by orders of magnitude in amplitude. A single global basis must split its capacity across all regimes, faithfully representing none.

We propose TEMPO (Trajectory EM Mixture of POD Operators), a two-phase framework for regime-aware operator learning. In the offline phase, TEMPO models the trajectory ensemble as a Gaussian mixture and recovers $M$ latent regimes via Expectation-Maximization: the E-step assigns each trajectory to regimes based on reconstruction quality, and the M-step fits a dedicated linear (POD) or nonlinear (NeuralPOD) basis to each. We prove that regime-local bases achieve reconstruction error no greater than the global rank-$K$ optimum for any soft assignment matrix and any $M$, $K$. In the online phase, a GatingNet and $M$ BranchNets are trained jointly, routing new inputs to the correct regime at inference.

We evaluate TEMPO on three benchmarks. On 1D Burgers' equation, TEMPO(POD) reduces error by 13–50% over the jointly-trained POD-DeepONet baseline across all viscosity values. On 2D Darcy flow, per-parameter specialists fail out-of-distribution, while TEMPO(POD) reduces error by up to 5×. On 2D Navier-Stokes, TEMPO matches the global POD baseline. The discovered regime partition recovers structure driven by both initial conditions and the physical parameter from solution trajectories, without regime-label supervision.

*Keywords:* operator learning, parametric PDE, regime discovery, proper orthogonal decomposition, expectation-maximization, neural operator

## 1. Introduction

Reduced-order modeling of parametric PDEs constructs low-rank bases from solution snapshots to enable fast prediction at new parameter values without

re-solving the governing equations [1]. The dominant approach, exemplified by POD-DeepONet [2], fits a single global basis to the full snapshot ensemble and trains a branch network to predict expansion coefficients for unseen inputs [3]. This approach succeeds when solutions lie on a smooth low-dimensional manifold, but fails when the governing parameter drives *qualitatively distinct* dynamics: viscosity separating laminar flow from near-discontinuous shocks, or source magnitude shifting solution amplitudes by orders of magnitude. A single global basis must distribute its capacity across all regimes, representing none faithfully.

Partitioning the parameter space and fitting local bases to each region is the classical remedy in reduced-order modeling [4, 5]. Daniel et al. [6] extend this to deep learning by clustering solution subspaces on the Grassmann manifold and training a dictionary selector at inference. These approaches reduce approximation error on multi-modal problems, but share a fundamental limitation: clustering is performed independently of basis quality, so the discovered partition need not align with the directions of greatest reconstruction benefit.

A second line of work enriches the basis while retaining a single global model. POD-PoU [7] blends several POD bases via a partition-of-unity function, reducing error on sharp-gradient problems. NeuralPOD [8] learns nonlinear mode functions by greedy residual minimisation, overcoming the linear subspace constraint of POD. Both improve representational capacity but treat the full ensemble jointly, without identifying distinct solution regimes.

Neural operators such as FNO [9] and CNO [10] learn solution operators as discretisation-invariant functional maps, achieving high accuracy without explicit basis decompositions. Their purpose is complementary to the present work: they do not produce inspectable basis representations or identify latent regime structure.

We propose **TEMPO** (*Trajectory EM Mixture of POD Operators*), which addresses this by coupling regime discovery directly with basis quality in a principled probabilistic framework. TEMPO models the trajectory ensemble as a Gaussian mixture and recovers regime structure via Expectation-Maximization (EM). The E-step assigns each trajectory to regimes based on reconstruction quality; the M-step fits a dedicated basis to each regime. Basis quality and regime assignments improve each other iteratively. After convergence, a GatingNet and per-regime BranchNets are trained jointly, routing new inputs to the correct regime at inference.

**Contributions:**

- **TEMPO framework.** A two-phase mixture-of-operators for parametric PDEs: an offline EM that discovers $M$ latent regimes from unlabelled snapshots and fits a dedicated orthogonal (POD) or nonlinear (NeuralPOD) basis to each; an online phase where a GatingNet and $M$ BranchNets are trained with a KL penalty that anchors routing to the offline regime partition.
- **Adaptive basis update.** A distribution-shift criterion that monitors moment changes in regime composition across EM iterations and selects per-regime between full retraining, incremental mode addition/pruning, or skipping.

- **Theoretical guarantee.** Regime-local bases achieve reconstruction error no greater than the global rank-$K$ optimum for any soft-assignment matrix and any $M$, $K$ (Proposition 1).
- **Experiments.** On 1D Burgers' and 2D Darcy flow, TEMPO(POD) reduces error by 13–50% over the jointly-trained POD-DeepONet baseline, while per-parameter specialists fail on unseen parameter values.

## 2. Problem Statement

Let $\Omega \subset \mathbb{R}^d$ be a bounded spatial domain and $\mathcal{T} \subset \mathbb{R}_{\geq 0}$ a time interval. We are given a dataset of $N$ solution trajectories:

$$\mathcal{D} = \left\{ (u_0^{(i)}, \kappa^{(i)}, s^{(i)}) \right\}_{i=1}^{N}, \qquad s^{(i)} = s\Big(\cdot, \cdot; u_0^{(i)}, \kappa^{(i)}\Big) \in L^2(\Omega \times \mathcal{T}), \quad (1)$$

where $u_0^{(i)} \in \mathcal{U} = L^2(\Omega)$ is the initial condition and $\kappa^{(i)} \in \mathcal{K}$ the physical parameter. Each trajectory $s^{(i)}$ is discretized on $N_y = N_t N_x$ quadrature points $\{y_j = (x_j, t_j), w_j\}_{j=1}^{N_y}$ covering the product grid $\Omega \times \mathcal{T}$, with weighted norm $\|f\|_w^2 = \sum_j w_j f(y_j)^2$.

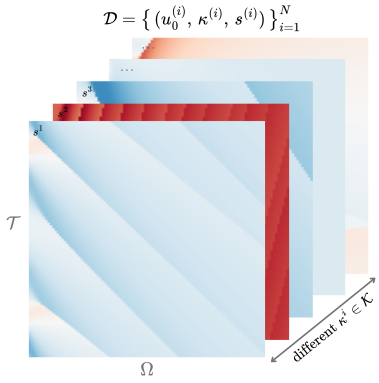



Figure 1: Training dataset $\mathcal{D}$: solution trajectories $s^{(i)}$ on $\Omega \times \mathcal{T}$ for different $\kappa^{(i)} \in \mathcal{K}$. Solutions change character fundamentally across parameter values.

**Definition 1** (Solution operator)**.** *The* solution operator

$$\mathcal{G} : \mathcal{U} \times \mathcal{K} \to L^2(\Omega \times \mathcal{T})$$

*maps each pair* $(u_0, \kappa)$ *to the full space-time solution:* $\mathcal{G}(u_0, \kappa) = s(\cdot, \cdot; u_0, \kappa)$.

The operator $\mathcal{G}$ is unknown and expensive to evaluate directly. The *operator learning problem* is to construct $\hat{\mathcal{G}} \approx \mathcal{G}$ from $\mathcal{D}$ alone, generalizing to unseen $(u_0, \kappa) \notin \mathcal{D}$ without solving the PDE.

*Multi-regime assumption.* When solutions change character fundamentally across $\mathcal{K}$, the optimal low-dimensional representation differs from regime to regime, and no single global basis captures the full solution diversity. We formalize this observation via a latent variable model over the trajectory ensemble.

**Definition 2** (Generative model)**.** *Each trajectory arises from one of $M$ latent regimes. The regime label $z_i \in \{1, \dots, M\}$ is unobserved and drawn from a categorical prior,*

$$z_i \sim \text{Categorical}(\pi_1, \dots, \pi_M), \qquad \pi_m \geq 0, \quad \textstyle\sum_m \pi_m = 1. \tag{2}$$

*Conditioned on $z_i = m$, the trajectory is distributed as*

$$(s^{(i)} \mid z_i{=}m) \;\sim\; \mathcal{N}\big(\hat{s}_m^{(i)},\; \sigma^2 W^{-1}\big), \tag{3}$$

*where $W = \text{diag}(w_1, \dots, w_{N_y})$ is the quadrature weight matrix, $\sigma^2 > 0$ controls the softness of regime assignments, and $\hat{s}_m^{(i)}$ is the regime-$m$ approximation to $s^{(i)}$.*

The generative model leaves $\hat{s}_m^{(i)}$ unspecified. We require only that it admits a *low-rank trajectory decomposition*:

$$\hat{s}_m^{(i)}(y) \;=\; \phi_0^{(m)}(y) + \sum_{k=1}^{K_m} \lambda_k^{(m,i)}\, \phi_k^{(m)}(y), \qquad y = (x,t) \in \Omega \times \mathcal{T}, \tag{4}$$

where $\phi_k^{(m)} : \Omega \times \mathcal{T} \to \mathbb{R}$ are spatiotemporal mode functions and $\lambda_k^{(m,i)} \in \mathbb{R}$ a scalar amplitude for trajectory $i$ on mode $k$. This form encompasses POD [2] and learned neural bases [8]; in this work we adopt the latter.

**Definition 3** (Regime approximator)**.** *The parameters of regime $m$ are $\Phi_m = \{\phi_k^{(m)}\}_{k=0}^{K_m}$ and scalar amplitudes $\Lambda_m = \{\lambda_k^{(m,i)}\}_{k=1,\,i=1}^{K_m,\,N}$, where $\phi_0^{(m)} : \Omega \times \mathcal{T} \to \mathbb{R}$ is the regime mean field, $\{\phi_k^{(m)}\}_{k\geq 1}$ spatiotemporal mode functions, $\lambda_k^{(m,i)} \in \mathbb{R}$ the scalar amplitude of trajectory $i$ on mode $k$, and $K_m \leq K_{\max}$ the adaptive mode count.*

The learning problem decomposes into two phases: discovering the latent regime structure from $\mathcal{D}$ *offline*, then training a deployable operator for fast inference *online*.

*Phase 1: Offline regime discovery.* Since the regime labels $z_i$ in (3) are unobserved, we recover the bases $\{\Phi_m\}$, mixture weights $\pi$, and noise level $\sigma^2$ by maximizing the observed-data log-likelihood:

$$(\Phi^*, \pi^*, \sigma^{2*}) \;=\; \underset{\Phi,\,\pi,\,\sigma^2}{\arg\max} \sum_{i=1}^{N} \log \sum_{m=1}^{M} \frac{\pi_m \, \det(W)^{1/2}}{(2\pi\sigma^2)^{N_y/2}} \exp\left( -\frac{\left\| s^{(i)} - \hat{s}_m^{(i)} \right\|_w^2}{2\sigma^2} \right). \tag{5}$$

The solution yields soft regime responsibilities $\gamma_{im}^* = p(z_i{=}m \mid s^{(i)}, \Phi^*, \pi^*, \sigma^{2*})$ that serve as supervision for Phase 2.

*Phase 2: Online operator learning.* With bases $\{\Phi_m^*\}$ and responsibilities $\{\gamma_{im}^*\}$ frozen, we train a gating network (*GatingNet*) and $M$ branch networks (*BranchNets*) that map a new input $(u_0, \kappa)$ to a weighted combination of local regime predictions. The precise architecture and loss are given in Section 3.4. The quality of the final operator is measured by the mean relative $L^2$ error:

$$\varepsilon \;=\; \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} \frac{\left\| s^{(i)} - \hat{s}(\cdot;\, u_0^{(i)}, \kappa^{(i)}) \right\|_w}{\left\| s^{(i)} \right\|_w}. \tag{6}$$

## 3. Method

All methods studied in this work share a common two-phase structure: an *offline* basis-fitting phase that constructs a low-rank approximation to the solution manifold from snapshots in $\mathcal{D}$, and an *online* operator-learning phase that trains a branch network to predict expansion coefficients from a new input $(u_0, \kappa)$. The four methods differ along two independent axes:

- **Basis type.** A *linear* (POD) basis uses SVD of the (weighted) snapshot matrix; a *nonlinear* (NeuralPOD) basis learns spatiotemporal mode networks by sequential residual minimization.
- **Number of regimes.** A *single-regime* operator fits one global basis to all snapshots; a *multi-regime* operator (TEMPO) discovers $M$ latent regimes via EM and assigns a dedicated basis to each.

This gives four combinations: POD-DeepONet [2] (linear, global), NeuralPOD-DeepONet (nonlinear, global, our extension of [8]), TEMPO(POD) (linear, $M$ regimes), and TEMPO(NeuralPOD) (nonlinear, $M$ regimes).

### *3.1. Basis Representations*

#### *3.1.1. POD Basis*

Let $\phi_0 := \bar{s} = \frac{1}{N}\sum_i s^{(i)}$ be the mean field and $\tilde{S} \in \mathbb{R}^{N_y \times N}$ the matrix with columns $\tilde{s}^{(i)} = s^{(i)} - \phi_0$. The rank-$K$ truncated SVD $\tilde{S} \approx U_K \Sigma_K V_K^\top$ yields orthonormal modes $\{\phi_k\}_{k=1}^K$ (columns of $U_K$) and coefficients $\lambda_k^{(i)} = [\Sigma_K V_K^\top]_{ki}$; the approximation

$$\hat{s}^{(i)} = \phi_0 + \sum_{k=1}^{K} \lambda_k^{(i)}\, \phi_k \tag{7}$$

is optimal in the $L^2$ sense among all rank-$K$ affine representations.

For TEMPO(POD), the M-step (17) is solved analytically: set $\phi_0^{(m)} := \bar{s}_m = \frac{1}{N_m}\sum_i \gamma_{im} s^{(i)}$, where $N_m = \sum_i \gamma_{im}$, and apply SVD to the weighted centered matrix with columns $\sqrt{\gamma_{im}}\,(s^{(i)} - \phi_0^{(m)})$. This is the closed-form linear analogue of Algorithm 1.

### 3.1.2. NeuralPOD Basis

To overcome the linearity constraint, we parametrize each mode function $\phi_k : \Omega \times \mathcal{T} \to \mathbb{R}$ as a Fourier feature network [11]; we call the resulting basis a *Fourier NeuralPOD* basis:

$$\phi_k(y) \;=\; f_{\theta_k}\big(\psi(y)\big), \qquad \psi(y) \;=\; \big[\sin(2\pi B y),\, \cos(2\pi B y)\big], \tag{8}$$

where $B \in \mathbb{R}^{F \times d}$ is a fixed random matrix drawn once at initialization and $f_{\theta_k} : \mathbb{R}^{2F} \to \mathbb{R}$ is a small MLP with Tanh activations. Plain MLPs exhibit spectral bias [12]; the Fourier encoding resolves this. For multi-scale coverage, the rows of $B$ are partitioned evenly across $L$ frequency scales $\sigma_1 < \cdots < \sigma_L$: the $\ell$-th block is drawn from $\mathcal{N}(0, \sigma_\ell^2 I)$. The mean field $\phi_0$ uses the same architecture.

The scalar amplitudes $\lambda_k^{(i)} \in \mathbb{R}$ are direct learnable parameters, one per training trajectory, optimized jointly with $\phi_k$. After Phase 1 they are frozen and serve as regression targets for the BranchNet in Phase 2.

Modes are learned by *greedy residual minimization* [8]. Let $r^{(0,i)} = s^{(i)} - \phi_0$, where $\phi_0$ is fitted to the snapshot mean $\bar{s}$ defined above. For each subsequent mode $k \geq 1$, given the residual

$$r^{(k-1,i)}(y) = s^{(i)}(y) - \phi_0(y) - \sum_{\ell=1}^{k-1} \lambda_\ell^{(i)}\, \phi_\ell(y), \tag{9}$$

we solve

$$\min_{\phi_k,\, \{\lambda_k^{(i)}\}} \sum_{i=1}^{N} \sum_{j} w_j \Big(\lambda_k^{(i)}\, \phi_k(y_j) - r_j^{(k-1,i)}\Big)^2 \tag{10}$$

(the global case with uniform weights; the $\gamma$-weighted generalization used in TEMPO is Algorithm 1). Mode addition stops when the residual falls below fraction $\tau$ of the initial variance,

$$\sum_{i=1}^{N} \|r^{(k,i)}\|_w^2 \;<\; \tau \cdot \sum_{i=1}^{N} \|s^{(i)} - \bar{s}\|_w^2, \tag{11}$$

or $k = K_{\max}$. Each mode captures the largest remaining variance component — the nonlinear analogue of Gram–Schmidt orthogonalization.

## 3.2. Single-Regime Operators

### 3.2.1. POD-DeepONet

POD-DeepONet [2] replaces the learned trunk network of DeepONet [3] with the fixed POD basis. After computing the global modes $\{\phi_k\}_{k=1}^K$ and training coefficients $\{\lambda^{(i)}\}$ offline, a branch network $\mathcal{B}_\theta : \mathbb{R}^{m_s + d_\kappa} \to \mathbb{R}^K$ is trained online by regression against these coefficients:

$$\mathcal{L}_{\text{branch}} = \frac{1}{N} \sum_{i=1}^{N} \big\| \mathcal{B}_\theta(u_0^{(i)},\, \kappa^{(i)}) - \lambda^{(i)} \big\|^2. \tag{12}$$

The prediction at any $(x,t) \in \Omega \times \mathcal{T}$ is

$$\hat{s}(x,t;\, u_0, \kappa) = \phi_0(x,t) + \sum_{k=1}^{K} \mathcal{B}_{\theta,k}(u_0,\kappa)\, \phi_k(x,t). \tag{13}$$

### *3.2.2. NeuralPOD-DeepONet*

Mou et al. [8] introduce NeuralPOD and pair it with a DeepONet branch network to predict mode coefficients for unseen inputs. We adopt this combination, which we refer to as *NeuralPOD-DeepONet*, as a single-regime baseline and as the per-regime basis learner within TEMPO(NeuralPOD).

The procedure follows POD-DeepONet with the Fourier NeuralPOD basis in place of POD. Offline fitting yields coefficients $\boldsymbol{\lambda}^{(i)} \in \mathbb{R}^K$; online, a branch network minimizes (12) and prediction follows (13). At equal mode count $K$, the nonlinear basis spans a strictly richer function space than POD.

## *3.3. TEMPO: Offline Regime Discovery*

A single global basis, whether linear or nonlinear, must compromise between qualitatively distinct regimes, as the optimal basis geometry differs fundamentally between them. TEMPO avoids this by constructing a dedicated basis for each of $M$ latent regimes, discovered from unlabelled trajectories via EM on the generative model of Definition 2.

*Initialization.* A global POD is computed on all $N$ trajectories; each trajectory is projected to a $d_0$-dimensional coefficient vector $\alpha^{(i)} \in \mathbb{R}^{d_0}$. Running a standard GMM on $\{\alpha^{(i)}\}$ yields initial responsibilities $\gamma_{im}^{(0)}$, weights $\pi_m^{(0)}$, and per-regime statistics $(\mu_m^{(0)}, \Sigma_m^{(0)})$. Each regime basis $\Phi_m^{(0)}$ is then fitted to the responsibility-weighted ensemble: for TEMPO(NeuralPOD) via NeuralBasis (Algorithm 1); for TEMPO(POD) via the weighted SVD of Section 3.1. The noise level $\sigma^2$ is calibrated from the initial reconstructions rather than treated as a free hyperparameter:

$$\sigma^2 \;\leftarrow\; \frac{1}{2N} \sum_{i=1}^{N} \sum_{m=1}^{M} \gamma_{im}^{(0)} \left\| s^{(i)} - \hat{s}_m^{(i)} \right\|_w^2, \tag{14}$$

$\sigma^2$ is held fixed throughout EM; improving bases reduce residuals, which progressively sharpen regime assignments.

*E-step.* At iteration $q$, the responsibility of trajectory $i$ for regime $m$ is updated by Bayes' rule on the Gaussian likelihood (3):

$$\gamma_{im}^{(q)} = \frac{\pi_m^{(q-1)} \, \exp\!\Big(-\dfrac{\|s^{(i)} - \hat{s}_m^{(i)}\|_w^2}{2\sigma^2}\Big)}{\sum_{j=1}^{M} \pi_j^{(q-1)} \, \exp\!\Big(-\dfrac{\|s^{(i)} - \hat{s}_j^{(i)}\|_w^2}{2\sigma^2}\Big)}, \tag{15}$$

where $\hat{s}_m^{(i)}$ is evaluated via (4) using $\Phi_m^{(q-1)}$ and $\Lambda_m^{(q-1)}$. As the bases improve in the M-step, the reconstructions $\hat{s}_m^{(i)}$ sharpen and the E-step automatically refines regime assignments in response — distinguishing TEMPO from a GMM applied to a fixed feature space. When $\kappa$ drives large amplitude variation (as with $\beta \in [0.01, 100]$ in the Darcy experiments), replacing the squared norm with the relative squared norm $\|s^{(i)} - \hat{s}_m^{(i)}\|_w^2 / \|s^{(i)}\|_w^2$ throughout (in (15), (14), and (17)) keeps E- and M-steps consistent under a heteroscedastic likelihood with noise variance proportional to $\|s^{(i)}\|_w^2$. When $\kappa$ is observed, the GMM initialization can operate in $\log \kappa$ space, and an optional $\kappa$-prior term in the E-step anchors assignments to the known parameter structure.

*M-step.* With responsibilities fixed, the M-step gives a closed-form update for the mixture weights,

$$\pi_m^{(q)} \leftarrow \frac{N_m^{(q)}}{N}, \qquad N_m^{(q)} = \sum_{i=1}^{N} \gamma_{im}^{(q)}, \tag{16}$$

and a weighted reconstruction problem for each basis:

$$\min_{\Phi_m,\, \Lambda_m} \sum_{i=1}^{N} \gamma_{im}^{(q)} \, \|s^{(i)} - \hat{s}_m^{(i)}\|_w^2, \tag{17}$$

solved by weighted SVD for TEMPO(POD) or NeuralBasis (Algorithm 1) for TEMPO(NeuralPOD). Since gradient descent solves the NeuralPOD sub-problem without guaranteeing global optimality, TEMPO is a Generalized EM algorithm [13], which guarantees that the observed-data log-likelihood is non-decreasing at every iteration.

**Proposition 1** (Regime Approximation Advantage)**.** *For any $\{\gamma_{im}\} \geq 0$ with $\sum_m \gamma_{im} = 1$, any $M \geq 1$, any $K \geq 1$:*

$$\varepsilon_{\text{TEMPO}}^K \;\leq\; \varepsilon_{\text{global}}^K,$$

*where*

$$\varepsilon_{\text{global}}^K = \min_{\substack{P \,\text{rank-}K \\ \mu}} \sum_i \|(I-P)(s^{(i)}-\mu)\|_w^2,$$

$$\varepsilon_{\text{TEMPO}}^K = \sum_m \min_{\substack{P_m \,\text{rank-}K \\ \mu_m}} \sum_i \gamma_{im} \|(I-P_m)(s^{(i)}-\mu_m)\|_w^2$$

*are the global and regime-wise optimal rank-$K$ reconstruction errors. Proof: Appendix A.1.*

**Remark 1.** *Both $\varepsilon_{\text{TEMPO}}^K$ and $\varepsilon_{\text{global}}^K$ use exactly $K$ basis functions per sample at inference; TEMPO stores $M$ regime-specific bases of rank $K$ offline, but routes each sample through exactly one.*

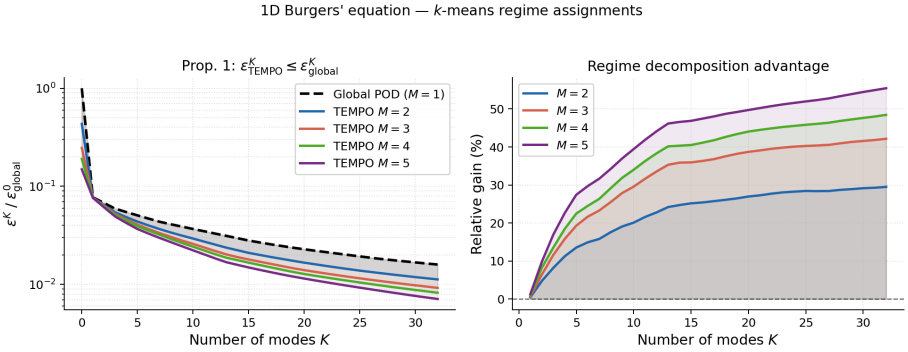



Figure 2: Empirical verification of Proposition 1 on 1D Burgers'. *Left*: normalized reconstruction error (log scale); TEMPO lies strictly below the global POD for all $K \geq 1$. *Right*: relative gain $(\varepsilon^K_{\text{global}} - \varepsilon^K_{\text{TEMPO}})/\varepsilon^K_{\text{global}}$; all values are non-negative, with gains of 20–39% at $K$=10.

*Adaptive basis update.* Retraining all $M$ NeuralPOD bases at every EM iteration is computationally prohibitive. We introduce a distribution-shift criterion to determine whether each basis requires updating. Let $\alpha^{(i)}$ denote the global POD projection of $s^{(i)}$, computed once during initialization and fixed thereafter. The responsibility-weighted first and second moments of regime $m$ are

$$\mu_m^{(q)} = \frac{1}{N_m^{(q)}} \sum_{i=1}^{N} \gamma_{im}^{(q)} \, \alpha^{(i)}, \tag{18}$$

$$\Sigma_m^{(q)} = \frac{1}{N_m^{(q)}} \sum_{i=1}^{N} \gamma_{im}^{(q)} \, (\alpha^{(i)} - \mu_m^{(q)})(\alpha^{(i)} - \mu_m^{(q)})^\top. \tag{19}$$

The distribution-shift criterion measures the relative change in these moments between consecutive iterations:

$$\Delta_m^{(q)} = \frac{\|\mu_m^{(q)} - \mu_m^{(q-1)}\|_2}{\|\mu_m^{(q-1)}\|_2 + \epsilon} + \frac{\|\Sigma_m^{(q)} - \Sigma_m^{(q-1)}\|_F}{\|\Sigma_m^{(q-1)}\|_F + \epsilon}. \tag{20}$$

where $\epsilon > 0$ is a small numerical stabilizer. $\Delta_m^{(q)}$ measures how much the composition of regime $m$ has changed, not just the volume of reassignments. Two trajectories with very different $\alpha^{(i)}$ swapping regimes produce a large $\Delta_m^{(q)}$ and trigger a basis update; two similar snapshots doing so do not. Given thresholds $0 < \varepsilon_s < \varepsilon_l$, regime $m$ is SKIPped if $\Delta_m^{(q)} < \varepsilon_s$, updated INCREMENTALly (add or prune one mode) if $\varepsilon_s \leq \Delta_m^{(q)} < \varepsilon_l$, and fully retrained from scratch (FULL RERUN) otherwise. In the INCREMENTAL case, we compute the residual of the current basis under the updated responsibilities $\gamma^{(q)}$, add a new mode if $\sum_i \gamma_{im}^{(q)} \|r_m^{(K_m,i)}\|_w^2 \geq \tau \cdot \sum_i \gamma_{im}^{(q)} \|s^{(i)} - \bar{s}_m^{(q)}\|_w^2$, and prune the last mode if its weighted contribution falls below $\varepsilon_p$. The FULL RERUN case uses a cold-start random initialization, as warm-starting may trap gradient descent near the previous regime's basis.

*Convergence and model selection.* The number of regimes $M$ is selected by two criteria: BIC on the EM log-likelihood (penalizes complexity) and mean assignment entropy $H$. When they conflict, entropy is decisive: a large gain $\Delta H$ from $M$ to $M{+}1$ signals a new regime. EM terminates when $\max_m \Delta_m^{(q)} < \varepsilon_c$. Selected values are $M{=}3$ (Burgers'), $M{=}4$ (Navier-Stokes), and $M{=}5$ (Darcy); details for Burgers' are in Table A.4 (Appendix A.3). The complete offline procedure is given in Algorithm 2. Figure 3 shows the resulting regime structure on 1D Burgers': three manifold branches emerge from unlabelled snapshots, one per discovered regime, with the partition reflecting joint variability from initial conditions and viscosity in the solution trajectories.

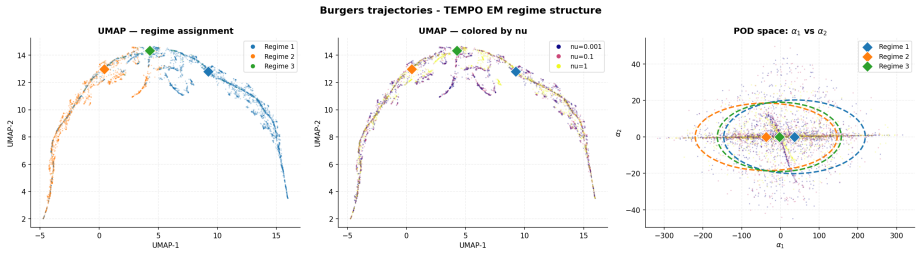



Figure 3: TEMPO(POD) regime structure on 1D Burgers' ($M{=}3$, 20 EM iterations). *Left:* UMAP colored by regime. *Center:* same embedding colored by $\nu$. *Right:* GMM fit in POD coefficient space $(\alpha_1, \alpha_2)$; stars mark regime centroids $\mu_m$.

### *3.4. TEMPO: Online Gated Operator*

After Algorithm 2 converges, all regime bases $\{\Phi_m^*\}$ and responsibilities $\{\gamma_{im}^*\}$ are frozen. The online phase trains a GatingNet $G : \mathbb{R}^{m_s+d_\kappa} \to \Delta^{M-1}$ that maps $(u_0, \kappa)$ to regime weights, and $M$ BranchNets $\mathcal{B}_m : \mathbb{R}^{m_s+d_\kappa} \to \mathbb{R}^{K_m}$ that predict expansion coefficients for each regime. The TEMPO prediction at query point $y = (x, t) \in \Omega \times \mathcal{T}$ is

$$\hat{s}(y;\, u_0, \kappa) = \sum_{m=1}^{M} g_m(u_0, \kappa) \left[ \phi_0^{(m)}(y) + \sum_{k=1}^{K_m} b_{m,k}(u_0, \kappa)\, \phi_k^{(m)}(y) \right], \tag{21}$$

where $\mathbf{g} = G(u_0, \kappa)$ and $\mathbf{b}_m = \mathcal{B}_m(u_0, \kappa)$. The BranchNet outputs $\mathbf{b}_m$ predict the expansion coefficients $\boldsymbol{\lambda}_m^{(i)}$ for new inputs $(u_0, \kappa)$ not seen in training. The prediction is mesh-free and evaluable at any $y \in \Omega \times \mathcal{T}$ without retraining.

*Training objective.* Writing $g_m^{(i)} = g_m(u_0^{(i)}, \kappa^{(i)})$, the online loss is

$$\begin{aligned}\mathcal{L}_{\text{online}} = & \underbrace{\frac{1}{N}\sum_{i=1}^{N} \|s^{(i)} - \hat{s}(\cdot;\, u_0^{(i)}, \kappa^{(i)})\|_w^2}_{\mathcal{L}_{\text{data}}} \\ & + \eta_{\text{KL}} \underbrace{\frac{1}{N}\sum_{i,m} g_m^{(i)} \log \frac{g_m^{(i)}}{\gamma_{im}^*}}_{\mathcal{L}_{\text{KL}}} \\ & - \eta_H \underbrace{\frac{1}{N}\sum_{i,m} g_m^{(i)} \log g_m^{(i)}}_{-\mathcal{L}_{\text{ent}}}.\end{aligned} \tag{22}$$

$\mathcal{L}_{\text{data}}$ drives reconstruction accuracy. $\mathcal{L}_{\text{KL}}$ anchors the online gating to the offline regime partition: it penalizes deviation of $\mathbf{g}^{(i)}$ from the EM responsibilities $\boldsymbol{\gamma}_i^*$, so the network cannot discard the discovered structure in pursuit of a simpler routing. $\mathcal{L}_{\text{ent}}$ prevents collapse to a single expert, ensuring all $M$ regime bases contribute. The hyperparameters $\eta_{\text{KL}}, \eta_H \geq 0$ trade off these objectives.

## 4. Computational Experiments

We evaluate TEMPO on parametric PDE benchmarks, verifying three claims: (i) the offline EM phase recovers interpretable, physically meaningful regime structure from unlabelled snapshots; (ii) locally adapted bases achieve lower reconstruction error than a single global basis at equal mode count; (iii) a gated operator with per-regime bases outperforms a single global basis when the data contains structured regimes.

### *4.1. Experimental Setup*

*Datasets.* We use three parametric PDE benchmarks; the first two are from PDEBench [14].

*1D Burgers' equation.*

$$u_t + u\,u_x = \nu\,u_{xx}, \qquad x \in (-1, 1), \quad t \in (0, 2], \tag{23}$$

with periodic boundary conditions and random initial conditions. Viscosity $\nu \in \{0.001, 0.01, 0.1, 1.0\}$ spans four qualitatively distinct solution regimes: smooth diffusive profiles at $\nu$=1.0 and near-discontinuous shock structures at $\nu$=0.001. Each value provides $N$=9,500 trajectories on $N_x$=1,024 spatial points and $N_t$=201 time steps (38,000 total).

*2D Darcy flow.*

$$-\nabla \cdot \big(a(x)\,\nabla u(x)\big) = \beta, \qquad x \in (0,1)^2, \tag{24}$$

with zero Dirichlet boundary conditions. Here $a(x)$ is a spatially varying permeability field sampled from a random sum of Gaussians, and $\beta$ is the constant source term taking five values: 0.01, 0.1, 1.0, 10.0, 100.0. Each value provides $N$=8,000 training and 1,000 test samples discretised on a 128×128 uniform grid ($N_y$=16,384 spatial points), 40,000 samples in total. The operator maps $a(x)$ to the pressure solution $u(x)$. As a steady-state problem, $N_t$=1 and there is no time dependence.

*2D incompressible Navier-Stokes.*

$$\partial_t \omega + \mathbf{u} \cdot \nabla \omega = \frac{1}{\mathrm{Re}}\,\Delta \omega + f, \qquad \mathbf{x} \in (0,1)^2,\ t \in (0,2], \tag{25}$$

with periodic boundary conditions, $\omega = \nabla \times \mathbf{u}$, divergence-free $\mathbf{u}$ recovered via the stream function, and Kolmogorov-type forcing $f(\mathbf{x}) = 0.1\big(\sin 2\pi(x_1{+}x_2) + \cos 2\pi(x_1{+}x_2)\big)$ [9]. Reynolds number $\mathrm{Re} \in \{100, 1000, 3600, 10000\}$ spans four qualitatively distinct regimes from smooth laminar flow to developed turbulence. Each value provides $N$=5,000 trajectories on a 64×64 periodic grid (20,000 total; 4,000/1,000 train/test); see Appendix A.2. Operator: $\mathbf{u}_0 \mapsto (\mathbf{u}_t)_{t=1}^{20}$, $\Delta t$=0.1.

*Methods.* We compare DeepONet [3], POD-DeepONet [2], FNO [9], and CNO [10], all trained jointly on all parameter values via a single global model. We introduce NeuralPOD-DeepONet (learned Fourier NeuralPOD basis with branch network); TEMPO(POD) and TEMPO(NeuralPOD) (regime-aware bases discovered via EM, routed by gating network). Per-parameter specialists provide in-distribution reference baselines. All methods implemented by the authors. Evaluation: mean relative $L^2$ error (6) per parameter value. Inference times measured on a single NVIDIA RTX A5000 GPU, per sample.

### *4.2. Main Results*

*1D Burgers' equation.* Table 1 gives the cross-viscosity error matrix. Per-$\nu$ specialists are unviable across the full range: a $\nu$=1.0 model degrades from 0.025 to 0.461 at $\nu$=0.001. TEMPO(POD) addresses this by training across all viscosity values, outperforming the joint POD-DeepONet baseline by 13–50%. TEMPO(NeuralPOD) does not surpass the joint baseline. NeuralPOD coefficients are harder to predict than POD projections, as seen already in the single-regime case (0.229 vs. 0.048 at $\nu$=0.1); iterative EM feedback amplifies the prediction difficulty.

*2D Darcy flow.* Table 2 gives the cross-$\beta$ matrix. Specialists fail out-of-distribution: Darcy solutions scale linearly with $\beta$, driving cross-parameter errors beyond 1000. TEMPO(POD) reduces error by 3–5× over the joint POD-DeepONet baseline (e.g., $\beta$=0.1: 0.775 vs. 2.557; $\beta$=100: 0.073 vs. 0.347). TEMPO(NeuralPOD) improves over the global NeuralPOD-DeepONet baseline ($\beta$=0.1: 3.813 vs. 72.9)

but does not match TEMPO(POD). FNO and CNO achieve lower absolute error without explicit basis structure; the interpretable regime decomposition that TEMPO provides is not available from these methods.

*2D Navier-Stokes.* Table 3 gives the cross-Re error matrix. TEMPO(POD) remains within 5–17% of the joint POD-DeepONet baseline but does not improve over it. Unlike Burgers' and Darcy, solutions at different Re share similar dominant POD directions: the EM assigns near-uniform regime weights, and local bases bring no reconstruction advantage over the global one. TEMPO(NeuralPOD) ($\kappa$-initialised, $M$=4) converges but does not close the gap to TEMPO(POD).

*Inference speed.* Routing each sample through a single regime basis keeps TEMPO(POD) fast. On Burgers', TEMPO(POD) requires 0.025 ms per sample versus 9.90 ms for FNO (400× faster) and 2.14 ms for CNO (86× faster); on Darcy, 0.006 ms versus 0.587 ms (FNO) and 2.24 ms (CNO). The overhead relative to POD-DeepONet is modest (0.025 vs. 0.004 ms on Burgers'), reflecting the additional gating network and $M$ branch evaluations of which only one is active per sample.

### *4.3. Phase 1 Reconstruction Advantage*

Figure 2 empirically verifies Proposition 1 on 1D Burgers' ($N$=1,500, $\nu \in \{0.001, 0.1, 1.0\}$) with $k$-means regime assignments. TEMPO achieves 20–39% lower reconstruction error than the global POD at $K$=10 modes, with the gap growing in both $M$ and $K$; no violations of the bound were observed across all $K \in \{1, \ldots, 32\}$.

### *4.4. Regime Discovery*

We select $M$=3 based on entropy gain: $H$ increases by +0.30 from $M$=2 to $M$=3 but only +0.18 from $M$=3 to $M$=4, indicating the third regime captures genuine structure that a fourth does not. BIC increases monotonically with $M$ (favoring $M$=2 by complexity alone), so entropy is the decisive criterion here (Table A.4, Appendix A.3). Figure 3 confirms three distinct manifold branches, one per discovered regime. The discovered regimes capture the joint structure of initial conditions and viscosity in the solution space. The per-viscosity gating weights (Table A.5, Appendix A.3) are nearly uniform across $\nu \in \{0.001, 0.1, 1.0\}$, closely matching the global weights $\boldsymbol{\pi}$=(0.324, 0.267, 0.409): the initial condition determines regime membership, while $\nu$ shapes within-regime dynamics. The same initial condition yields a shock at $\nu$=0.001 and a smooth wave at $\nu$=1.0. At inference, $\nu$ is passed to the branch network as a conditioning input, capturing both sources of solution variability.

### *4.5. Role of the Physical Parameter*

We ablate the effect of $\kappa$ at inference by replacing it with zero or a learned prediction. On Burgers', TEMPO(NeuralPOD) barely needs $\kappa$: removing it costs only ∼1% in L2, since $u_0$ carries sufficient information to determine regime routing on this benchmark. TEMPO(POD) is more sensitive (+37%); however, $\nu$ can be predicted from $u_0$ with 87% accuracy, reducing the gap to +7%.

Table 1: **Relative $L^2$ test error on 1D Burgers' equation.** Each per-$\nu$ specialist is trained on a single viscosity and evaluated on all three jointly-trained values; underlined entries are in-distribution. The large off-diagonal errors motivate joint training with regime discovery. All jointly-trained methods use $\nu \in \{0.001, 0.1, 1.0\}$. **Bold**: best per column among jointly-trained methods.

| Method | Train $\nu$ | $\nu$=0.001 | $\nu$=0.1 | $\nu$=1.0 | Infer. (ms) |
|---|---|---|---|---|---|
| *Per-$\nu$ specialists (single-viscosity training)* | | | | | |
| DeepONet [3] | 0.001 | 0.536 | 0.474 | 0.382 | 0.037 |
| | 0.1 | 0.535 | 0.471 | 0.349 | |
| | 1.0 | 0.547 | 0.483 | 0.283 | |
| POD-DeepONet [2] | 0.001 | 0.273 | 0.297 | 0.639 | 0.004 |
| | 0.01 | 0.274 | 0.274 | 0.614 | |
| | 0.1 | 0.349 | 0.048 | 0.344 | |
| | 1.0 | 0.461 | 0.260 | 0.025 | |
| NeuralPOD-DeepONet *(ours)* | 0.001 | 0.392 | 0.288 | 0.535 | 3.53 |
| | 0.01 | 0.391 | 0.284 | 0.541 | |
| | 0.1 | 0.426 | 0.229 | 0.371 | |
| | 1.0 | 0.483 | 0.315 | 0.183 | |
| FNO [9] | 0.001 | 0.426 | 0.529 | 0.623 | 9.90 |
| | 0.01 | 0.454 | 0.401 | 0.492 | |
| | 0.1 | 0.650 | 0.304 | 0.521 | |
| | 1.0 | 1.161 | 0.919 | 0.158 | |
| CNO [10] (2.0M) | 0.01 | 0.687 | 0.955 | 2.344 | 2.14 |
| | 0.1 | 0.983 | 0.199 | 0.471 | |
| | 1.0 | 1.870 | 1.084 | 0.125 | |
| *Joint training ($\nu \in \{0.001, 0.1, 1.0\}$)* | | | | | |
| DeepONet [3] | all | 0.533 | 0.472 | 0.289 | 0.037 |
| POD-DeepONet [2] | all | 0.460 | 0.339 | 0.210 | **0.004** |
| FNO [9] | all | 0.427 | 0.298 | 0.202 | 9.90 |
| CNO [10] (2.0M) | all | 0.361 | 0.162 | **0.133** | 2.14 |
| TEMPO(POD) *(ours)* | $M$=2 | 0.319 | **0.150** | 0.159 | 0.025 |
| | $M$=3 | **0.317** | 0.168 | 0.182 | |
| TEMPO(NeuralPOD) *(ours)* | $M$=2 | 0.536 | 0.458 | 0.326 | 0.024 |
| | $M$=3 | 0.542 | 0.456 | 0.288 | |

Table 2: **Relative $L^2$ test error on 2D Darcy flow.** Each per-$\beta$ specialist is trained on a single source value and evaluated on all five; underlined entries are in-distribution. Off-diagonal errors exceed 1 because the Darcy solution scales with $\beta$, so a specialist trained on $\beta$=100 predicts amplitudes $\sim 10^4$ times larger than the ground truth at $\beta$=0.01. TEMPO trains jointly on $\beta \in \{0.1, 1, 10, 100\}$; the $\beta$=0.01 regime is excluded from joint training (—). **Bold**: best per column among jointly-trained methods.

| Method | Train $\beta$ | $\beta$=0.01 | $\beta$=0.1 | $\beta$=1.0 | $\beta$=10 | $\beta$=100 | Infer. (ms) |
|---|---|---|---|---|---|---|---|
| *Per-$\beta$ specialists (single-value training)* | | | | | | | |
| DeepONet [3] | 0.01 | 4.197 | 0.615 | 0.944 | 0.994 | 0.999 | 0.025 |
| | 0.1 | >10 | 0.851 | 0.875 | 0.987 | 0.999 | |
| | 1.0 | >10 | >1 | 0.433 | 0.902 | 0.972 | |
| | 10 | >100 | >10 | >1 | 0.243 | 0.705 | |
| | 100 | >1000 | >100 | >10 | >1 | 0.078 | |
| POD-DeepONet [2] | 0.01 | 0.449 | 0.791 | 0.961 | 0.996 | >1 | 0.001 |
| | 0.1 | >1 | 0.148 | 0.871 | 0.986 | 0.999 | |
| | 1.0 | >10 | >1 | 0.048 | 0.897 | 0.990 | |
| | 10 | >100 | >10 | >1 | 0.035 | 0.900 | |
| | 100 | >1000 | >100 | >10 | >1 | 0.042 | |
| NeuralPOD-DeepONet *(ours)* | 0.01 | 1.407 | 0.786 | 0.962 | 0.996 | >1 | 0.168 |
| | 0.1 | >1 | 0.421 | 0.902 | 0.989 | 0.999 | |
| | 1.0 | >10 | >1 | 0.135 | 0.901 | 0.990 | |
| | 10 | >100 | >10 | >1 | 0.050 | 0.900 | |
| | 100 | >1000 | >100 | >10 | >1 | 0.037 | |
| FNO [9] | 0.01 | 0.529 | 0.791 | 0.967 | 0.997 | 0.9998 | 0.587 |
| | 0.1 | 6.26 | 0.144 | 0.877 | 0.988 | 0.999 | |
| | 1.0 | 73.1 | 8.41 | 0.034 | 0.909 | 0.992 | |
| | 10 | >100 | 72.6 | 8.18 | 0.013 | 0.900 | |
| | 100 | >1000 | 256.3 | 43.2 | 6.13 | 0.008 | |
| CNO [10] (2.0M) | 0.01 | 0.388 | 0.929 | 1.000 | 1.001 | 1.000 | 2.24 |
| | 0.1 | 10.6 | 0.125 | 0.948 | 1.004 | 1.002 | |
| | 1.0 | 67.0 | 7.84 | 0.024 | 0.896 | 0.989 | |
| | 10 | >1000 | 99.1 | 8.37 | 0.009 | 0.898 | |
| *Joint training ($\beta \in \{0.1, 1, 10, 100\}$)* | | | | | | | |
| DeepONet [3] | all | — | 76.53 | 8.503 | 1.352 | 0.493 | 0.025 |
| POD-DeepONet [2] | all | — | 2.557 | 0.363 | 0.181 | 0.347 | **0.001** |
| NeuralPOD-DeepONet *(ours)* | all | — | 72.9 | 6.92 | 0.357 | 0.345 | 0.168 |
| FNO [9] | all | — | 0.380 | 0.083 | 0.023 | 0.008 | 0.587 |
| CNO [10] (2.0M) | all | — | **0.133** | **0.026** | **0.008** | **0.005** | 2.24 |
| TEMPO(POD) *(ours)* | $M$=5 | — | 0.775 | 0.178 | 0.136 | 0.073 | 0.006 |
| TEMPO(NeuralPOD) *(ours)* | $M$=5 | — | 3.813 | 0.549 | 0.322 | 0.311 | 0.006 |

Table 3: **Relative $L^2$ test error on 2D incompressible Navier-Stokes.** Each per-Re specialist is trained on a single Reynolds number and evaluated on all four; underlined entries are in-distribution. Joint methods train on all Re simultaneously. **Bold**: best per column among jointly-trained methods. † kappa-initialised variant.

| Method | Train Re | Re=100 | Re=1000 | Re=3600 | Re=10000 | Infer. (ms) |
|---|---|---|---|---|---|---|
| *Per-Re specialists (single-Re training)* | | | | | | |
| DeepONet [3] | 100 | 1.000 | 1.002 | 1.003 | 1.004 | 0.135 |
| | 1000 | 1.000 | 1.000 | 1.000 | 1.000 | |
| | 3600 | 1.002 | 1.000 | 1.000 | 1.000 | |
| | 10000 | 1.001 | 1.000 | 1.000 | 1.000 | |
| POD-DeepONet [2] | 100 | 0.082 | 0.748 | 0.865 | 0.877 | 0.005 |
| | 1000 | 0.667 | 0.129 | 0.671 | 0.812 | |
| | 3600 | 0.660 | 0.392 | 0.180 | 0.623 | |
| | 10000 | 0.934 | 0.687 | 0.556 | 0.345 | |
| NeuralPOD-DeepONet *(ours)* | 1000 | 0.762 | 0.198 | 0.607 | 0.761 | 4.87 |
| FNO [9] | 100 | 0.013 | 0.262 | 0.368 | 0.437 | 0.30 |
| | 1000 | 0.287 | 0.033 | 0.138 | 0.227 | |
| | 3600 | 0.381 | 0.150 | 0.039 | 0.128 | |
| | 10000 | 0.461 | 0.260 | 0.126 | 0.042 | |
| CNO [10] | 100 | 0.014 | 0.047 | 0.055 | 0.058 | 0.50 |
| | 1000 | 0.027 | 0.028 | 0.033 | 0.035 | |
| | 3600 | 0.030 | 0.029 | 0.033 | 0.035 | |
| | 10000 | 0.030 | 0.028 | 0.032 | 0.034 | |
| *Joint training (all Re)* | | | | | | |
| POD-DeepONet [2] | all | 0.092 | 0.132 | 0.141 | 0.145 | **0.005** |
| FNO [9] | all | **0.009** | **0.019** | **0.024** | **0.027** | 0.30 |
| NeuralPOD-DeepONet *(ours)* | all | 0.700 | 0.713 | 0.314 | 0.320 | 4.87 |
| CNO [10] | all | 0.020 | 0.035 | 0.040 | 0.043 | 0.50 |
| TEMPO(POD) *(ours)* | $M$=4 | 0.097 | 0.144 | 0.155 | 0.170 | 0.021 |
| TEMPO(NeuralPOD)† *(ours)* | $M$=4 | 0.595 | 0.620 | 0.694 | 0.627 | 0.021 |

On Darcy, $\kappa$ is essential for TEMPO(POD): removing $\beta$ collapses gating to a fixed mixture, increasing error by $\sim 3\times$. TEMPO(NeuralPOD) is less sensitive ($+27\%$), as its nonlinear basis partially compensates. In both cases, $\beta$ cannot be predicted from $a(x)$, as it is drawn independently in the dataset (classifier accuracy 21%, below chance). Full results are in Table A.6 (Appendix A.3).

## 5. Conclusion

Operator learning for parametric PDEs typically relies on a single global representation. When the governing parameter drives qualitative change in solution structure, a single global basis cannot fit all regimes well and prediction quality suffers.

TEMPO discovers regime structure from solution trajectories via EM and fits a dedicated basis to each regime. The discovered partition captures variability driven by both initial conditions and the physical parameter. On 1D Burgers', regimes persist across all viscosity values: the same initial condition produces a shock at $\nu$=0.001 and a smooth wave at $\nu$=1.0, yet falls in the same regime throughout, with $\nu$ shaping dynamics within each regime. A global model cannot separate these sources of variation; TEMPO recovers their structure from trajectory data without regime-label supervision.

TEMPO(POD) outperforms the joint POD-DeepONet baseline by 13–50% on Burgers' and up to $5\times$ on Darcy flow, where per-parameter specialists fail to generalize. On Navier-Stokes, solutions at different Re share similar POD directions, and TEMPO correctly recovers the global baseline without imposing spurious structure. TEMPO(NeuralPOD) extends the framework to nonlinear bases; improving nonlinear basis fitting within the EM loop remains an open direction.

Several directions remain open. Conditioning basis functions on $\kappa$ would let each mode encode parameter-dependent dynamics directly. Extending regime discovery to continuous parameter spaces and three-dimensional problems is a natural next step.

## References


[1] N. B. Kovachki, S. Lanthaler, A. M. Stuart, Operator learning: Algorithms and analysis, arXiv (2024). arXiv:2402.15715.

[2] L. Lu, X. Meng, S. Cai, Z. Mao, S. Goswami, Z. Zhang, G. E. Karniadakis, A comprehensive and fair comparison of two neural operators (with practical extensions) based on fair data, arXiv (2022). arXiv:2111.05512.

[3] L. Lu, P. Jin, G. E. Karniadakis, Deeponet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators, arXiv (2019). arXiv:1910.03193.

[4] D. Amsallem, M. J. Zahr, C. Farhat, Nonlinear model order reduction based on local reduced-order bases, International Journal for Numerical Methods in Engineering 92 (10) (2012) 891–916. doi:10.1002/nme.4371.

[5] M. Hess, A. Alla, A. Quaini, G. Rozza, M. Gunzburger, A localized reduced-order modeling approach for PDEs with bifurcating solutions, Computer Methods in Applied Mechanics and Engineering 351 (2019) 379–403. doi:10.1016/j.cma.2019.03.050.

[6] T. Daniel, F. Casenave, N. Akkari, D. Ryckelynck, Model order reduction assisted by deep neural networks (ROM-net), Advanced Modeling and Simulation in Engineering Sciences 7 (1) (2020) 16. doi:10.1186/s40323-020-00153-6.

[7] R. Sharma, V. Shankar, Ensemble and mixture-of-experts deeponets for operator learning, arXiv (2025). arXiv:2405.11907.

[8] C. Mou, B. Lu, G. Lin, Neural-pod: A plug-and-play neural operator framework for infinite-dimensional functional nonlinear proper orthogonal decomposition, arXiv (2026). arXiv:2602.15632.

[9] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, A. Anandkumar, Fourier neural operator for parametric partial differential equations, in: International Conference on Learning Representations, 2021.

[10] B. Raonić, R. Molinaro, J. Mishler, T. Rohner, F. Bartolucci, R. Alaifari, S. Mishra, E. de Bézenac, Convolutional neural operators for robust and accurate learning of PDEs, in: Advances in Neural Information Processing Systems, Vol. 36, 2023.

[11] M. Tancik, P. P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. T. Barron, R. Ng, Fourier features let networks learn high frequency functions in low dimensional domains, in: Advances in Neural Information Processing Systems, Vol. 33, 2020, pp. 7537–7547.

[12] N. Rahaman, A. Baratin, D. Arpit, F. Draxler, M. Lin, F. A. Hamprecht, Y. Bengio, A. Courville, On the spectral bias of neural networks, in: Proceedings of the 36th International Conference on Machine Learning, Vol. 97 of Proceedings of Machine Learning Research, 2019, pp. 5301–5310.

[13] R. M. Neal, G. E. Hinton, A view of the EM algorithm that justifies incremental, sparse, and other variants, in: M. I. Jordan (Ed.), Learning in Graphical Models, Vol. 89 of NATO ASI Series, Springer, Dordrecht, 1998, pp. 355–368. doi:10.1007/978-94-011-5014-9_12.

[14] M. Takamoto, T. Praditia, R. Leiteritz, D. MacKinlay, F. Alesiani, D. Pflüger, M. Niepert, Pdebench: An extensive benchmark for scientific machine learning, arXiv (2022). arXiv:2210.07182.

## Appendix A.

### *Appendix A.1. Proof of Proposition 1*

**Lemma 1.** *Let $\bar{s} = \frac{1}{N}\sum_i s^{(i)}$ be the global mean and $\bar{s}_m = \frac{1}{N_m}\sum_i \gamma_{im} s^{(i)}$ the regime-$m$ mean, with $N_m = \sum_i \gamma_{im}$. For any rank-$K$ orthoprojector $P$ and any regime $m$,*

$$\sum_i \gamma_{im}\|(I-P)(s^{(i)}-\bar{s}_m)\|_w^2 = \sum_i \gamma_{im}\|(I-P)(s^{(i)}-\bar{s})\|_w^2 - N_m\|(I-P)(\bar{s}_m-\bar{s})\|_w^2.$$

*Proof.* Let $c_m = \bar{s}_m - \bar{s}$. Writing $(I-P)(s^{(i)}-\bar{s}_m) = (I-P)(s^{(i)}-\bar{s}) - (I-P)c_m$ and expanding $\|a-b\|_w^2 = \|a\|_w^2 - 2\langle a,b\rangle_w + \|b\|_w^2$, then summing over $i$ with weights $\gamma_{im}$:

$$\begin{aligned}\sum_i \gamma_{im}\|(I-P)(s^{(i)}-\bar{s}_m)\|_w^2 = &\sum_i \gamma_{im}\|(I-P)(s^{(i)}-\bar{s})\|_w^2 \\ &- 2\Big\langle (I-P)\sum_i \gamma_{im}(s^{(i)}-\bar{s}),\ (I-P)c_m\Big\rangle_w \\ &+ N_m\|(I-P)c_m\|_w^2.\end{aligned}$$

Since $\sum_i \gamma_{im}(s^{(i)}-\bar{s}) = N_m c_m$, the cross-term equals $-2N_m\|(I-P)c_m\|_w^2$; together with the quadratic term this gives $-N_m\|(I-P)c_m\|_w^2 = -N_m\|(I-P)(\bar{s}_m-\bar{s})\|_w^2$. □

*Proof of Proposition 1.* Let $P^*$ and $P_m^*$ be the globally and regime-$m$ optimal rank-$K$ projectors. Since $P_m^*$ minimizes the regime-$m$ objective over all rank-$K$ projectors, substituting the (generally suboptimal) $P^*$ cannot decrease it:

$$\varepsilon_{\mathrm{TEMPO}}^K = \sum_m\sum_i \gamma_{im}\|(I-P_m^*)(s^{(i)}-\bar{s}_m)\|_w^2 \ \le\ \sum_m\sum_i \gamma_{im}\|(I-P^*)(s^{(i)}-\bar{s}_m)\|_w^2.$$

Applying Lemma 1 to each $m$ and summing over $m$, then using $\sum_m \gamma_{im} = 1$ for each $i$:

$$\begin{aligned}\sum_m\sum_i \gamma_{im}\|(I-P^*)(s^{(i)}-\bar{s}_m)\|_w^2 &= \sum_i \|(I-P^*)(s^{(i)}-\bar{s})\|_w^2 - \underbrace{\sum_m N_m\|(I-P^*)(\bar{s}_m-\bar{s})\|_w^2}_{\ge 0} \\ &\le \sum_i \|(I-P^*)(s^{(i)}-\bar{s})\|_w^2 = \varepsilon_{\mathrm{global}}^K.\end{aligned}$$

□

### *Appendix A.2. 2D Navier-Stokes Data Generation*

We use the same equation and forcing as in [9] with a pseudo-spectral solver in stream-function formulation: diffusion treated implicitly (Crank–Nicolson), advection explicitly (second-order Adams–Bashforth), and $\frac{2}{3}$-rule dealiasing on

the 64×64 grid. Simulation time-step $\delta t$=0.01; each recorded snapshot aggregates 10 sub-steps ($\Delta t$=0.1). Initial vorticity fields are random sums of sinusoids,

$$\omega_0(\mathbf{x}) = \sum_{k=1}^{7} \frac{a_k}{1+k}\big[\sin(2\pi k\, x_1+\phi_k)+\cos(2\pi k\, x_2+\phi_k)\big], \quad a_k \sim \mathcal{N}(0,1),\ \phi_k \sim \mathcal{U}[0,2\pi].$$

A 5 s burn-in (500 simulation steps) discards transient behavior before recording begins.

*Appendix A.3. Additional Experiments*

*Regime model selection (1D Burgers').* Table A.4 reports BIC, mean assignment entropy $H$, and mean test error $\bar{\varepsilon}$ for $M \in \{2,3,4\}$. Table A.5 shows the per-viscosity EM responsibilities alongside global mixture weights $\pi_m$.

Table A.4: Regime count selection on 1D Burgers'. BIC $= -2\log\mathcal{L} + k\log N$ (lower is better; penalizes complexity); $H$: mean assignment entropy (higher gain signals a new genuine regime); $\bar{\varepsilon}$: mean relative $L^2$ test error. BIC increases monotonically, favoring $M$=2 by complexity alone. **Bold row**: selected configuration ($M$=3), chosen because the entropy gain $\Delta H$=0.30 from $M$=2→3 substantially exceeds $\Delta H$=0.18 from $M$=3→4.

| $M$ | BIC ($\times 10^4$, ↓) | $H$ | $\bar{\varepsilon}$ |
|---|---|---|---|
| 2 | 3.90 | 0.622 | 0.209 |
| **3** | **4.92** | **0.920** | 0.222 |
| 4 | 5.47 | 1.099 | 0.228 |

Table A.5: Mean EM assignment weights per viscosity (1D Burgers', TEMPO(POD), $M$=3) vs. global mixture weights $\pi_m$. Each row sums to 1.

| $\nu$ | Regime 1 | Regime 2 | Regime 3 |
|---|---|---|---|
| 0.001 | 0.32 | 0.28 | 0.40 |
| 0.1 | 0.32 | 0.26 | 0.42 |
| 1.0 | 0.33 | 0.26 | 0.41 |
| $\pi_m$ | 0.32 | 0.27 | 0.41 |

*Role of the physical parameter (ablation).* Table A.6 reports the effect of replacing $\kappa$ with zero or a learned prediction at inference.

*POD vs. NeuralPOD basis comparison (1D Burgers').* Figure A.4 shows the singular-value and residual decay for POD ($K$=30 modes capture 99.99% of the variance) and Fourier NeuralPOD. The corresponding spatiotemporal mode functions (Figure A.5) reveal a structural difference: POD modes are globally supported, while NeuralPOD modes localise adaptively — at $\nu$=0.01 they concentrate near the shock front. Per-sample reconstructions from both bases are compared in Figure A.7; additional NeuralPOD examples are shown in Figure A.6.

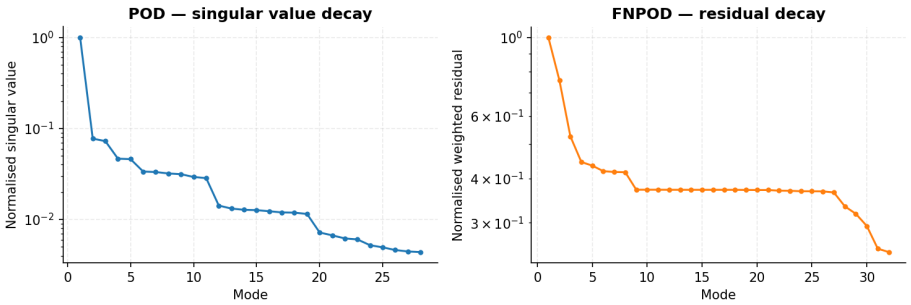
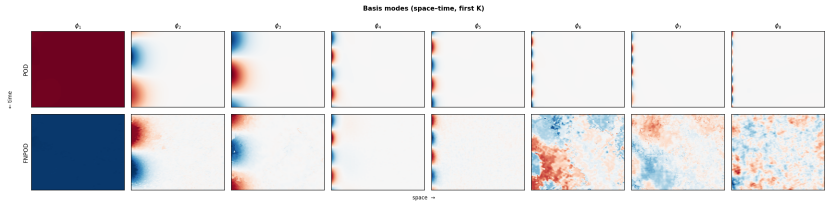
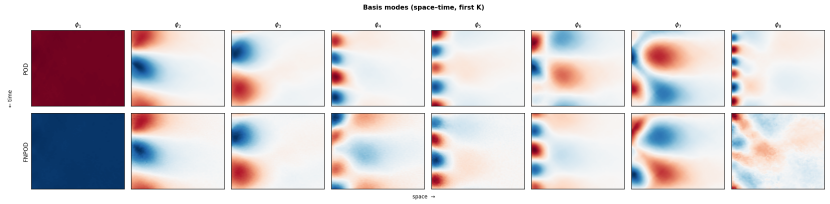



Figure A.4: Basis convergence on 1D Burgers' ($\nu$=1.0): POD singular-value decay (*left*); Fourier NeuralPOD residual decay (*right*).




(a) $\nu$=1.0




(b) $\nu$=0.01

Figure A.5: First $K$ basis modes on 1D Burgers': POD (*top row*) vs. Fourier NeuralPOD (*bottom row*).

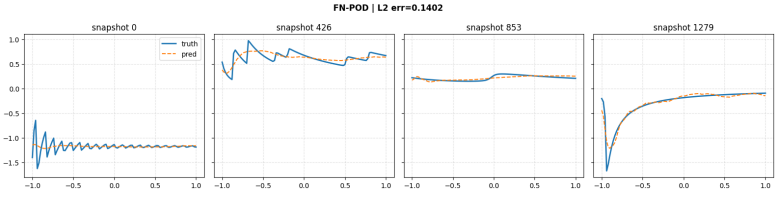



Figure A.6: Fourier NeuralPOD representative reconstructions on 1D Burgers' ($\nu$=1.0).

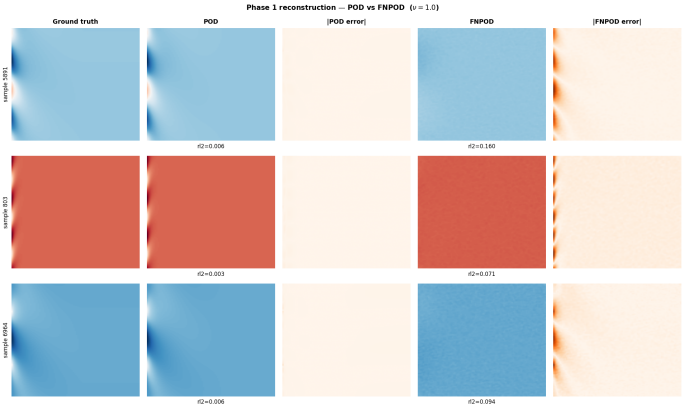
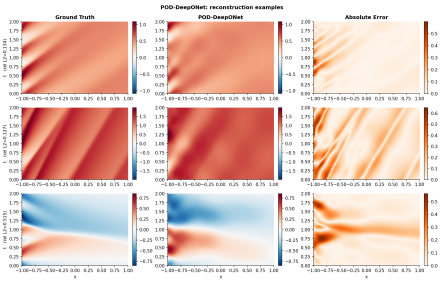
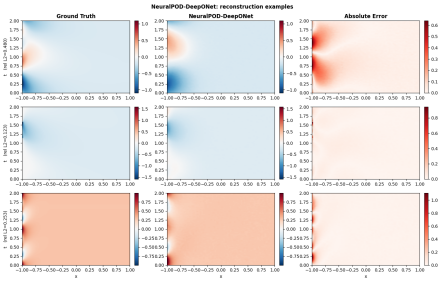


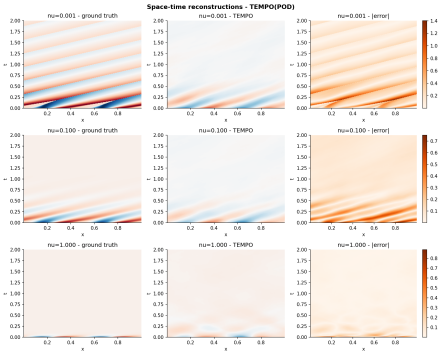

Figure A.7: Phase 1 reconstruction comparison on 1D Burgers' ($\nu$=1.0). Each row shows one sample: ground truth, POD reconstruction, |POD error|, Fourier NeuralPOD reconstruction, |FNPOD error|.



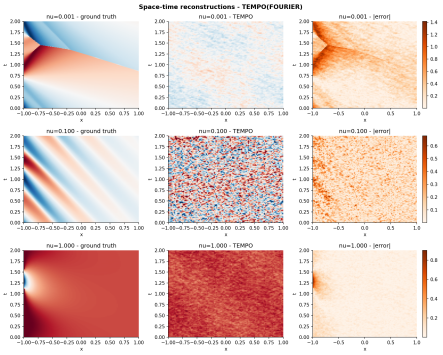

(a) POD-DeepONet




(b) NeuralPOD-DeepONet




(c) TEMPO(POD), $M$=3




(d) TEMPO(NeuralPOD), $M$=3: high-frequency noise (L2 0.46–0.54)

Figure A.8: Space–time reconstructions on 1D Burgers' ($\nu$=1.0).

Table A.6: $\kappa$ ablation: increase in mean relative L2 error when $\kappa$ is removed or predicted, relative to the real-$\kappa$ baseline. For Darcy, $\beta$ is drawn independently of $a(x)$; prediction from input data is not possible.

| | Burgers' | | Darcy | |
|---|---|---|---|---|
| | NPOD ($M$=3) | POD ($M$=3) | NPOD ($M$=5) | POD ($M$=5) |
| $\kappa = 0$ | +0.006 (+1%) | +0.127 (+37%) | +0.334 (+27%) | +0.618 (+219%) |
| $\kappa$ predicted from $u_0$ | — | +0.024 (+7%) | not predictable | |

*Representative reconstructions (1D Burgers').* Figure A.8 shows space–time predictions for all four methods on 1D Burgers' ($\nu$=1.0). Each panel: ground truth (*left*), prediction (*center*), point-wise error (*right*).

*TEMPO regime structure across benchmarks.* Figure A.10 compares the Phase 1 regime structure across all three benchmarks. On Burgers' (NeuralPOD), the partition mirrors the POD variant (Figure 3) with sharper assignments. On Darcy flow, regimes span multiple $\beta$ values; TEMPO(NeuralPOD) finds a nearly identical partition. On Navier-Stokes, all Re values overlap in POD coefficient space and regimes 2–4 collapse to a few points, explaining why dedicated bases bring no gain.

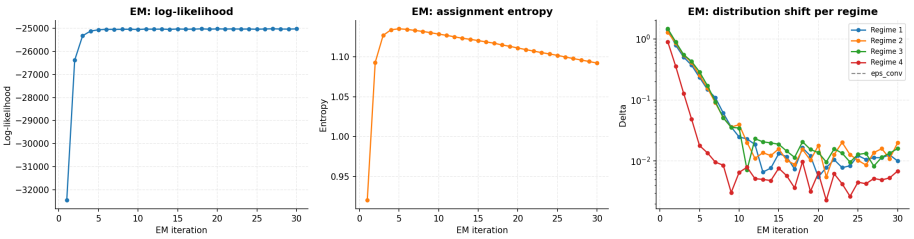
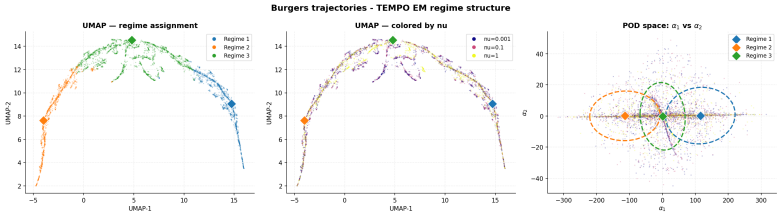


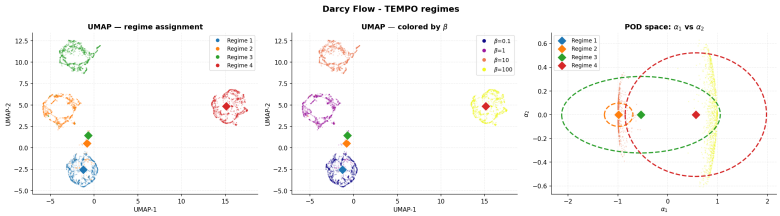

Figure A.9: TEMPO(POD) EM convergence on 2D Navier-Stokes ($M$=4).

*TEMPO reconstructions (2D Darcy flow).* Figure A.11 shows representative predictions from both TEMPO variants on 2D Darcy flow ($M$=5). Each panel: ground truth (*left*), prediction (*center*), point-wise error (*right*).

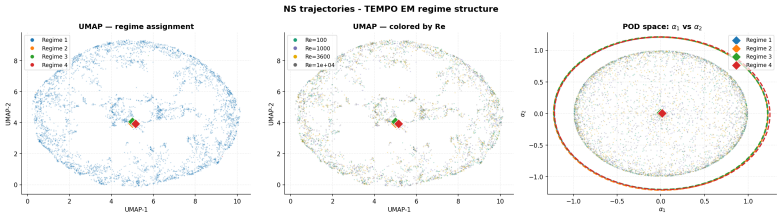
*TEMPO reconstructions (2D Navier-Stokes).* Figure A.12 shows representative predictions from TEMPO(POD) and TEMPO(NeuralPOD) on 2D Navier-Stokes ($M$=4); EM convergence is shown in Figure A.9.




(a) 1D Burgers', NeuralPOD ($M$=3, $H$=0.23)

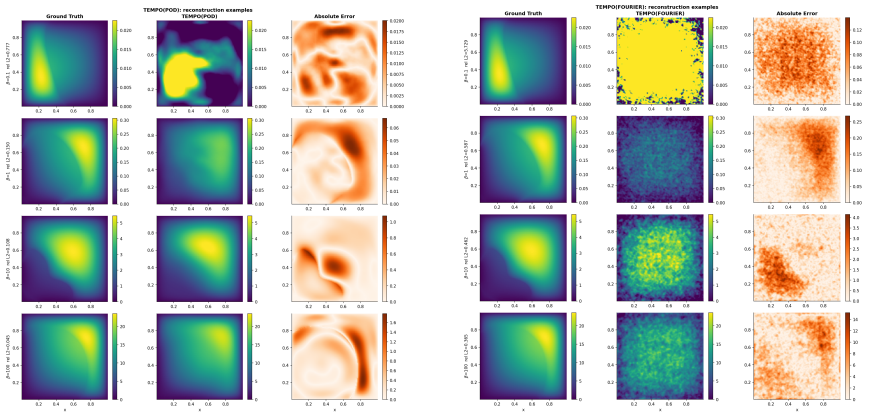



(b) 2D Darcy flow, POD ($M$=4)



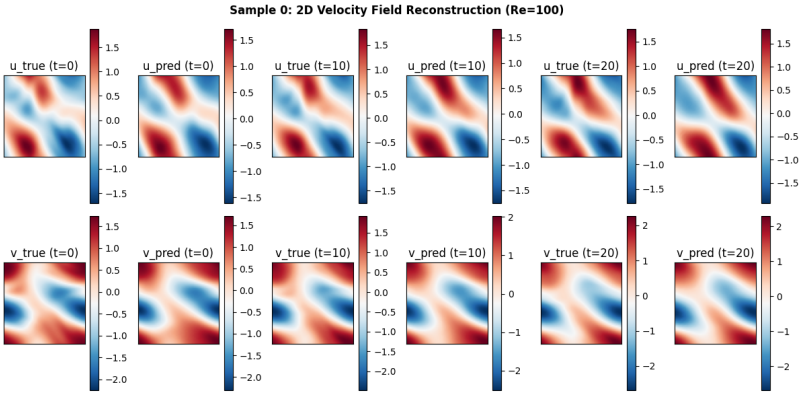

(c) 2D Navier-Stokes, POD ($M$=4): no Re separation

Figure A.10: TEMPO EM regime structure across all benchmarks. Layout as in Figure 3.

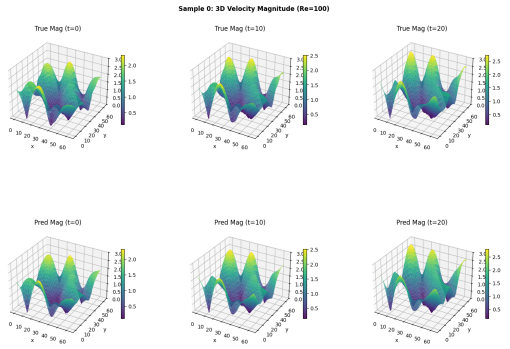



(a) TEMPO(POD) (b) TEMPO(NeuralPOD)

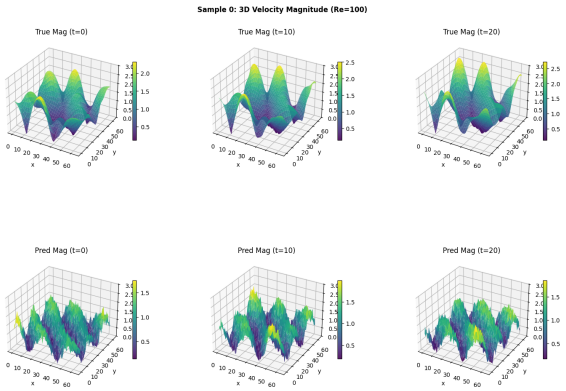
Figure A.11: Space–time reconstructions on 2D Darcy flow ($M$=5).




(a) TEMPO(POD): ground truth (*left*), prediction (*center*), point-wise absolute error (*right*).




(b) TEMPO(POD) — 3D vorticity field.




(c) TEMPO(NeuralPOD) — 3D vorticity field.

Figure A.12: 2D Navier-Stokes ($M$=4): TEMPO(POD) prediction (*top*); TEMPO(POD) and TEMPO(NeuralPOD) 3D vorticity (*middle*, *bottom*).

**Algorithm 1** NeuralBasis

**Require:** weighted snapshots $\{s^{(i)},\, \gamma_{im}\}_{i=1}^{N}$, tolerance $\tau$
**Ensure:** basis $\phi_0^{(m)},\ \{\phi_k^{(m)},\, \{\lambda_k^{(m,i)}\}_{i=1}^{N}\}_{k=1}^{K_m}$

1: $\bar{s}^{(m)} \leftarrow \dfrac{1}{N_m}\displaystyle\sum_{i=1}^{N} \gamma_{im}\, s^{(i)}, \quad N_m = \sum_{i=1}^{N} \gamma_{im}$ ▷ responsibility-weighted mean trajectory
2: Train $\phi_0^{(m)}$ by minimizing $\displaystyle\sum_j w_j \big(\phi_0^{(m)}(y_j) - \bar{s}_j^{(m)}\big)^2$
3: $r^{(0,i)} \leftarrow s^{(i)} - \phi_0^{(m)}(y)$ for all $i$; $k \leftarrow 0$
4: $V_m \leftarrow \displaystyle\sum_{i=1}^{N} \gamma_{im}\, \|s^{(i)} - \bar{s}^{(m)}\|_w^2$ ▷ initial unmodeled variance
5: **while** $\displaystyle\sum_{i=1}^{N} \gamma_{im}\, \|r^{(k,i)}\|_w^2 \;\geq\; \tau\, V_m$ **and** $k < K_{\max}$ **do**
6: Jointly minimize over $\phi_{k+1}^{(m)}$ and $\{\lambda_{k+1}^{(m,i)}\}_{i=1}^{N}$:

$$\sum_{i=1}^{N} \gamma_{im} \sum_j w_j \Big(\lambda_{k+1}^{(m,i)}\, \phi_{k+1}^{(m)}(y_j) - r_j^{(k,i)}\Big)^2$$

7: $r^{(k+1,i)} \leftarrow r^{(k,i)} - \lambda_{k+1}^{(m,i)}\, \phi_{k+1}^{(m)}(y)$ for all $i$
8: $k \leftarrow k + 1$
9: **end while**
10: **return** $K_m \leftarrow k,\ \phi_0^{(m)},\ \{\phi_k^{(m)},\, \{\lambda_k^{(m,i)}\}_{i=1}^{N}\}_{k=1}^{K_m}$

**Algorithm 2** TEMPO

**Require:** $\mathcal{D} = \{u_0^{(i)}, \kappa^{(i)}, s^{(i)}\}_{i=1}^{N}$; $M$, $\varepsilon_s$, $\varepsilon_l$, $\varepsilon_p$, $\varepsilon_c$, $\tau$
**Ensure:** Regime bases $\{\Phi_m^*\}$, amplitudes $\{\Lambda_m^*\}$, responsibilities $\{\gamma_{im}^*\}$, weights $\{\pi_m^*\}$

*Initialization*
Compute global POD on $\{s^{(i)}\}$; project to $\alpha^{(i)} \in \mathbb{R}^{d_0}$ for all $i$
Run GMM-EM on $\{\alpha^{(i)}\} \to \gamma_{im}^{(0)}$, $\pi_m^{(0)}$, $\mu_m^{(0)}$, $\Sigma_m^{(0)}$
**for** $m = 1, \dots, M$ **do**
    $\Phi_m^{(0)}, \Lambda_m^{(0)} \leftarrow$ NeuralBasis($\{s^{(i)}, \gamma_{im}^{(0)}\}, \tau$) ▷ or weighted SVD for TEMPO(POD), see §3.1
**end for**
$\sigma^2 \leftarrow (2N)^{-1} \sum_{i,m} \gamma_{im}^{(0)} \, \|s^{(i)} - \hat{s}_m^{(i)}\|_w^2$ (14)

*EM iterations*
**repeat**
  *E-step*
  **for** all $i = 1, \dots, N$ and $m = 1, \dots, M$ **do**
      Compute $\hat{s}_m^{(i)}$ via (4) using $\Phi_m^{(q-1)}$ and $\Lambda_m^{(q-1)}$
      $\gamma_{im}^{(q)} \leftarrow$ Eq. (15)
  **end for**
  *M-step: mixture weights*
  $\pi_m^{(q)} \leftarrow N_m^{(q)}/N$ for all $m$ (16)
  *M-step: bases*
  **for** $m = 1, \dots, M$ **do**
      Compute $\mu_m^{(q)}$ (18), $\Sigma_m^{(q)}$ (19), $\Delta_m^{(q)}$ (20)
      **if** $\Delta_m^{(q)} < \varepsilon_s$ **then**
          $\Phi_m^{(q)} \leftarrow \Phi_m^{(q-1)}$ Skip
      **else if** $\Delta_m^{(q)} < \varepsilon_l$ **then** Incremental
          Compute $r_m^{(K_m)}$ under updated $\gamma^{(q)}$; $V_m^{(q)} \leftarrow \sum_i \gamma_{im}^{(q)} \|s^{(i)} - \bar{s}_m^{(q)}\|_w^2$
          **if** $\sum_i \gamma_{im}^{(q)} \|r_m^{(K_m,i)}\|_w^2 \geq \tau \cdot V_m^{(q)}$ **then**
              Train mode $K_m{+}1$; $K_m \leftarrow K_m + 1$
          **end if**
          **if** $\|\phi_{K_m}^{(m)}\|_w^2 \cdot \sum_{i=1}^{N} \gamma_{im}^{(q)} \left(\lambda_{K_m}^{(m,i)}\right)^2 < \varepsilon_p$ **then**
              $K_m \leftarrow K_m - 1$
          **end if**
      **else** Full rerun
          $\Phi_m^{(q)}, \Lambda_m^{(q)} \leftarrow$ NeuralBasis($\{s^{(i)}, \gamma_{im}^{(q)}\}, \tau$)
      **end if**
  **end for**
**until** $\max_m \Delta_m^{(q)} < \varepsilon_c$
**return** $\Phi_m^* \leftarrow \Phi_m^{(q)}$, $\Lambda_m^* \leftarrow \Lambda_m^{(q)}$, $\gamma_{im}^* \leftarrow \gamma_{im}^{(q)}$, $\pi_m^* \leftarrow \pi_m^{(q)}$